

Code of the Day

```
#include <omp.h>
#include <stdio.h>
int main() {
    int N = 12;
    int a = 0;
    double start = omp_get_wtime();
    #pragma omp parallel
    {
        #pragma omp single
        std::printf("# threads: %d\n", omp_get_num_threads());

        for (int i = 0; i < N; i++) {
            a++;
        }
    }
    double end = omp_get_wtime();
    std::printf("Work took: %f seconds\n", end - start);
    std::printf("Result: a = %d\n", a);
    return 0;
}
```

Is the following outcome guaranteed?

```
$ g++ -std=c++17 -O3 test.cpp -fopenmp
$ OMP_NUM_THREADS=2 ./a.out
# threads: 2
Work took: 0.000197 seconds
Result: a = 24
$ OMP_NUM_THREADS=3 ./a.out
Number of threads: 3
Work took: 0.000198 seconds
Result: a = 36
$ OMP_NUM_THREADS=4 ./a.out
Number of threads: 4
Work took: 0.000185 seconds
Result: a = 48
```

ECE 759

High Performance Computing for Applications in Engineering

[Spring 2025]

OpenMP – Loop Scheduling

02/21/2025

Before we get started...

- Quick overview, things discussed last time
 - OpenMP basics: what OpenMP is, how OpenMP is used, when to and not to use OpenMP
 - #pragma statement: A directive to instruct the compiler to generate parallel code
 - Runtime API: A set of runtime functions to alter OpenMP parameters at runtime
 - Environment variables: A key-value pair
 - The concept of “structured block” – a code region enclosed by {} to be affected by OpenMP constructs
 - Parallel-for construct and nested parallelism
- Purpose of today’s lecture:
 - Work sharing in OpenMP – the parallel for construct and scheduling algorithms
- Miscellaneous
 - Assignment #3 is released – due on **Monday 3/3 at 23:50 PM**

Work Plan, OpenMP

Parallel Regions

Work Sharing

Variable Scoping Issues

Synchronization

Performance Issues

Problem of Work Sharing

```
#include <omp.h>
#include <stdio.h>
int main() {
    int N = 12;
    int a = 0; // by default, any captured variables are
               // shared by OpenMP threads
    double start = omp_get_wtime();
    #pragma omp parallel
    {
        #pragma omp single
        std::printf("# threads: %d\n", omp_get_num_threads());

        for (int i = 0; i < N; i++) {
            a++; // a is shared and thus has data race
        }
    }
    double end = omp_get_wtime();
    std::printf("Work took: %f seconds\n", end - start);
    std::printf("Result: a = %d\n", a);
    return 0;
}
```

Due to data race, output is meaningless

```
$ g++ -std=c++17 -O3 test.cpp -fopenmp
$ OMP_NUM_THREADS=2 ./a.out
# threads: 2
Work took: 0.000197 seconds
Result: a = 13
$ OMP_NUM_THREADS=3 ./a.out
Number of threads: 3
Work took: 0.000198 seconds
Result: a = 23
$ OMP_NUM_THREADS=4 ./a.out
Number of threads: 4
Work took: 0.000185 seconds
Result: a = 11
```

Data Race

- A **data race** occurs when two or more threads access the same memory location at the same time, and at least one of those accesses is a write, without proper synchronization
 - By default, OpenMP captures data by reference (shared scope)
- Data race can lead to unpredictable behavior because the threads may interfere with each other's operations

```
#pragma omp parallel num_threads(4)
{
    #pragma omp single
    std::printf("# threads: %d\n", omp_get_num_threads());

    for (int i = 0; i < N; i++) {
        a++; // a is raced by four threads
    }
}
```

One Solution to Avoid Data Race: OpenMP **atomic** Directive

- The **atomic** directive protects a single assignment with exclusive access on its memory
 - The term “atomic” refers to an operation that is indivisible, meaning it either completes fully or not at all, without any interference from other operations
 - Unlike `std::mutex`, `atomic` is *lock-free* and directly implemented through hardware-specific ISA
- The memory updates that can be protected by **atomic** belongs to one of the following:

```
x++;
```

```
++x;
```

```
x--;
```

```
--x;
```

```
x <op>= <expression>;
```

<op> can be one of: +, *, -, /, &, ^, |, <<, >>

- <expression> must not reference x
- Only the load and store of x is protected (not <expression>)

Use OpenMP **atomic** to avoid Data Race during Work Sharing

```
#include <omp.h>
#include <stdio.h>
int main() {
    int N = 12;
    int a = 0;
    double start = omp_get_wtime();
    #pragma omp parallel
    {
        #pragma omp single
        std::printf("# threads: %d\n", omp_get_num_threads());

        for (int i = 0; i < N; i++) {
            #pragma omp atomic
            a++; // makes the update on "a" atomic
        }
    }
    double end = omp_get_wtime();
    std::printf("Work took: %f seconds\n", end - start);
    std::printf("Result: a = %d\n", a);
    return 0;
}
```

With atomic, we can see the following expected output:

```
$ g++ -std=c++17 -O3 test.cpp -fopenmp
$ OMP_NUM_THREADS=2 ./a.out
# threads: 2
Work took: 0.000197 seconds
Result: a = 24
$ OMP_NUM_THREADS=3 ./a.out
Number of threads: 3
Work took: 0.000198 seconds
Result: a = 36
$ OMP_NUM_THREADS=4 ./a.out
Number of threads: 4
Work took: 0.000185 seconds
Result: a = 48
```

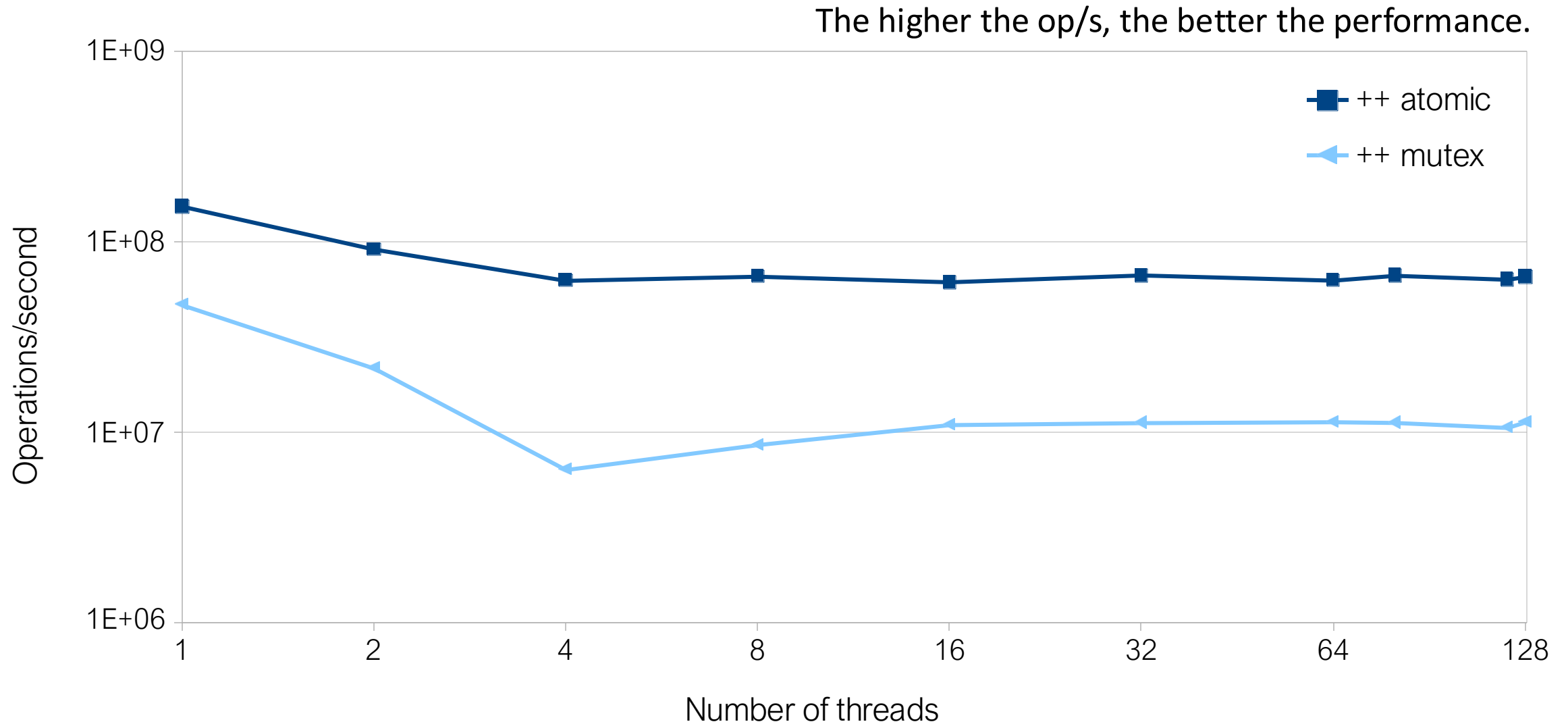

Under the Hood of OpenMP **atomic**

- OpenMP **atomic** is in general implemented using C++ [std::atomic](#)
 - C++ `std::atomic` is a template that defines a sequences of instructions guaranteeing *atomic* accesses and updates of plain-old data (POD) type variables without the need of explicit lock
 - C++ `std::atomic` provides many methods for performing atomic operations
 - Atomic sub/add
 - Atomic bitwise and/or

```
int sum(0);
std::mutex mutex;
#omp parallel num_threads(4)
{
    mutex.lock()
    sum = sum + 1;
    mutex.unlock();
}
```

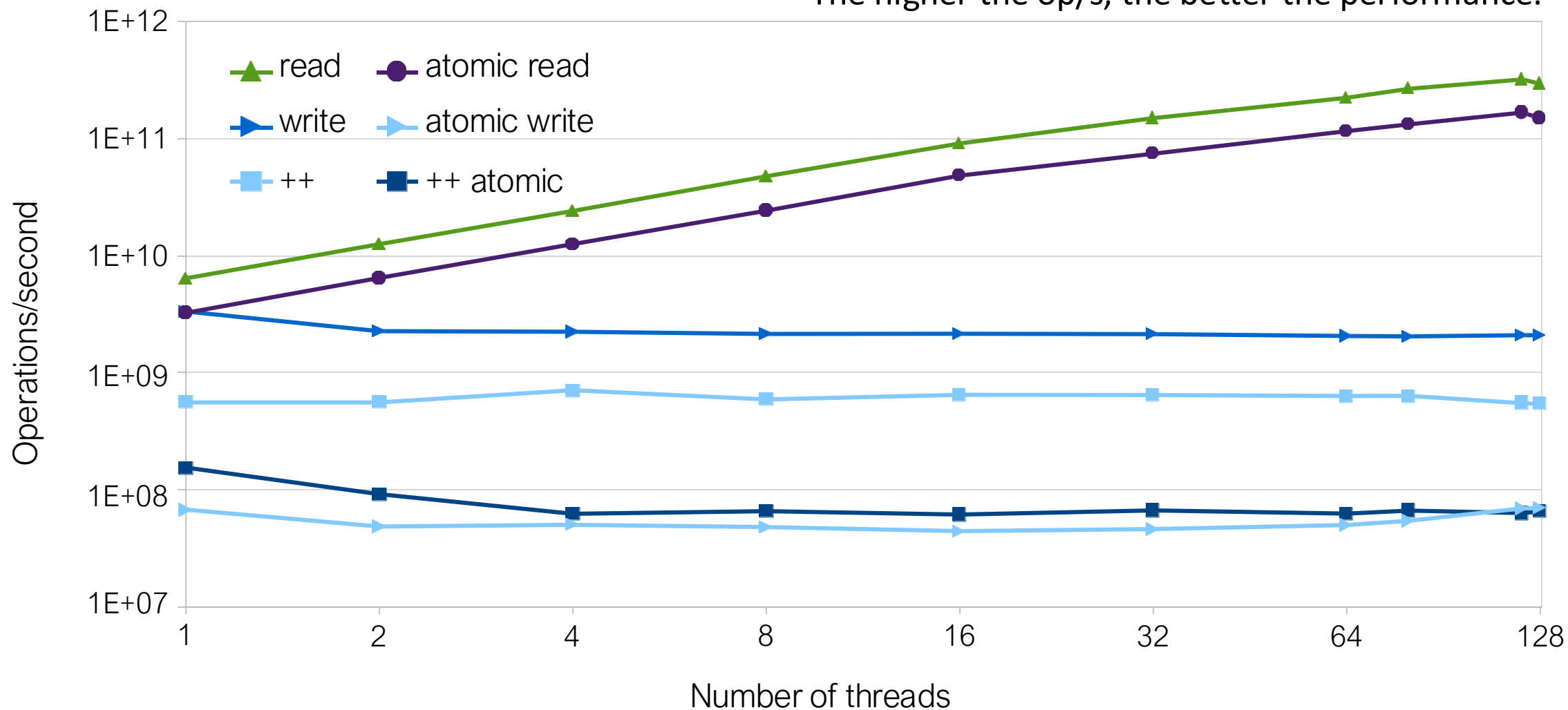
```
std::atomic<int> sum(0);
#omp parallel num_threads(4)
{
    sum.fetch_add(1); // sum = sum + 1
}
```

Performance: Atomic Operations vs Lock-based Operations



The Cost of Atomic Operations

The higher the op/s, the better the performance.



`std::atomic` is Great, but Not All Types can be Made Atomic

- What types can be atomic?
 - Any trivially copyable type can be made atomic
 - However, the implementation may not always be lock-free
- A trivially copyable type has the following properties:
 - Continuous chunk of memory
 - Copying the object means copying all bits (`std::memcpy`)
 - No virtual functions, noexcept constructor
 - Basically, most plain-old-data (POD) types can be made atomic
- In addition to arithmetic add/sub, `std::atomic<T>` support many other operations:

```
std::atomic<int> i;  
std::atomic<float> x;  
struct S { long x; long y; };  
std::atomic<S> S;
```

```
struct A { int a[100]; };  
struct B { int x, y; };  
std::atomic<A>{}.is_lock_free(); // 0  
std::atomic<B>{}.is_lock_free(); // 1
```

```
x.load();           // atomically: T y = x  
x.store(y);         // atomically: x = y  
x.exchange(y);      // atomically: z=x; x=y
```

```
x.compare_exchange_strong(y, z);  
// atomically do the following:  
// if x==y, make x=z and return true  
// other, set y=x and return false  
// aka compare-and-swap (CAS) operation
```

What's So Special and Powerful about Compare-and-Swap (CAS)?

- Compare-and-swap (CAS) is a fundamental building blocks of many *lock-free* algorithms
 - Lock-free queue/stack, concurrent vector, etc.
- CAS allows you to support all kinds of arithmetic operations and accomplish them atomically
 - For `int`, we have atomic increment (`++`, `--`), but CAS can be used to increment doubles, multiply integers, and many more
 - Ex: Use CAS to implement atomic `++`

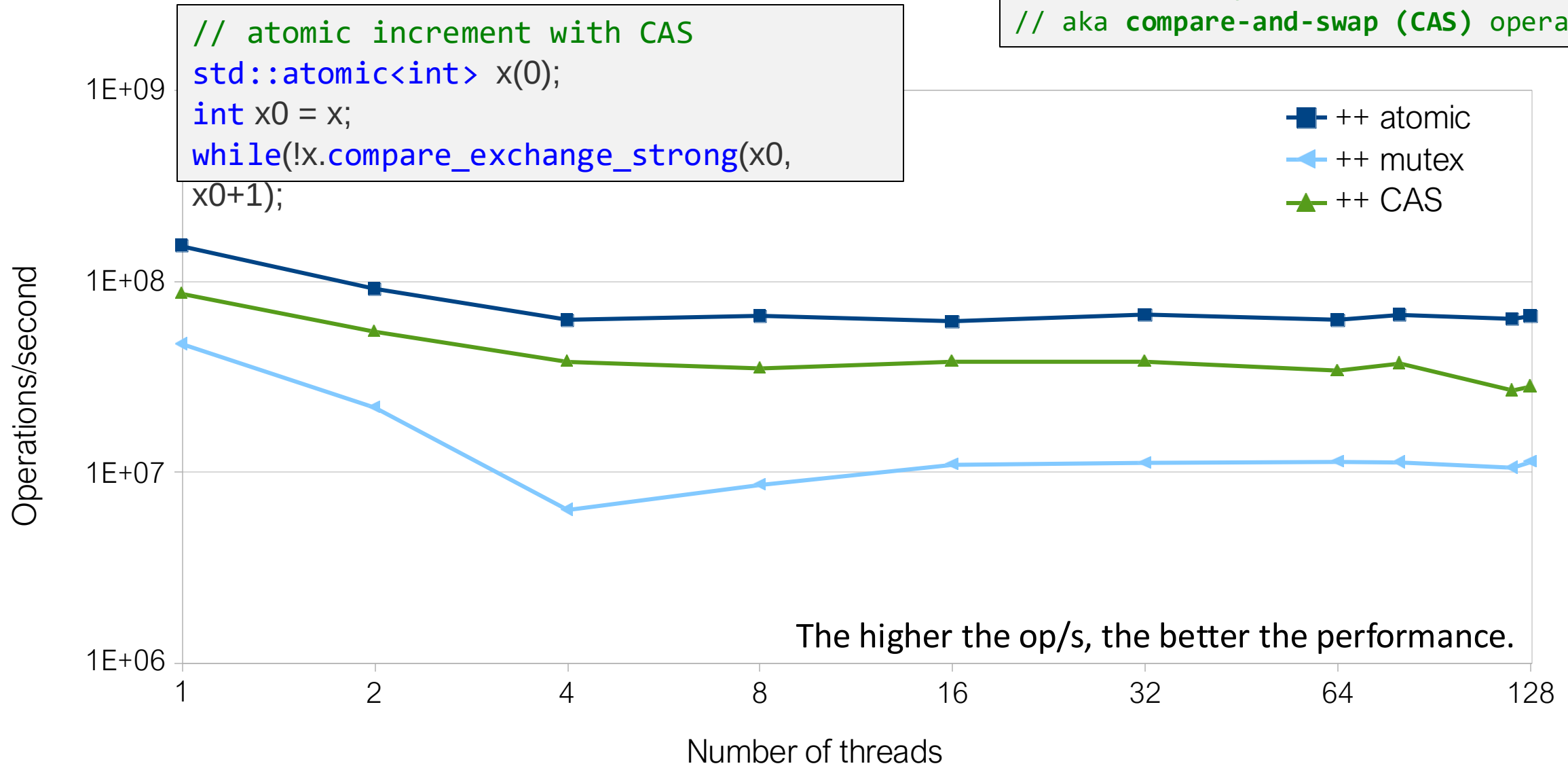
```
std::atomic<int> x(0);  
int x0 = x;  
while(!x.compare_exchange_strong(x0, x0+1);
```

- Ex: Use CAS to implement multiplication

```
while(!x.compare_exchange_strong(x0, x0*2);
```

Performance: CAS vs Atomic Increment

```
x.compare_exchange_strong(y, z);  
// atomically do the following:  
// if x==y, make x=z and return true  
// other, set y=x and return false  
// aka compare-and-swap (CAS) operation
```



Quiz: Can the Assertion fail?

```
std::atomic<int> counter(0);
void increment() {
    int expected=counter.load(), desired=expected+1;
    while (!counter.compare_exchange_strong(expected, desired));
}
int main() {
    std::thread t1(increment);
    std::thread t2(increment);
    t1.join();
    t2.join();
    assert(counter.load() == 2);
    return 0;
}
```

Quiz: Can the Assertion fail?

```
std::atomic<int> counter(0);
void increment() {
    int expected=counter.load(), desired=expected+1;
    while (!counter.compare_exchange_strong(expected, desired)) {
        desired = expected + 1;
    }
}
int main() {
    std::thread t1(increment);
    std::thread t2(increment);
    t1.join();
    t2.join();
    assert(counter.load() == 2);
    return 0;
}
```


Learn More about C++ Atomics

YouTube Home

cppcon 
the c++ conference
SEPTEMBER 24-29 2017
Bellevue, Washington, USA

**C++ atomics: from basic to advanced.
What do they really do?**

Presenter: Fedor Pikus

<https://www.youtube.com/watch?v=ZQFzMfHlxng>

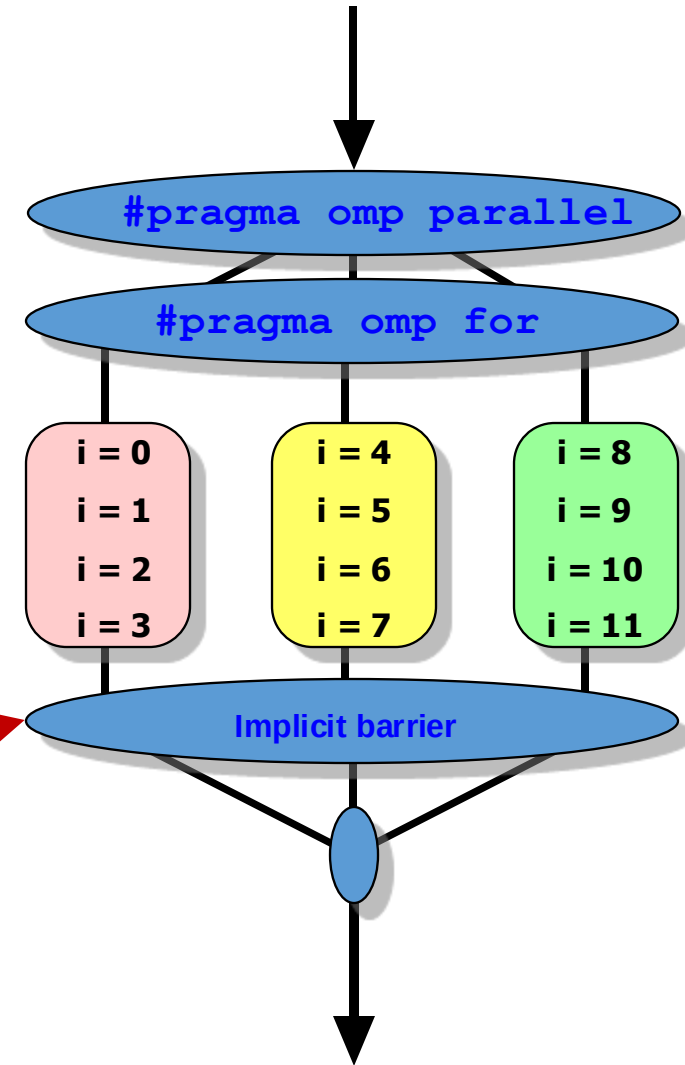
Work Sharing in OpenMP

- **Work sharing**: general term used in OpenMP to describe distribution of work across threads
 - Three primary constructs for work sharing in OpenMP:
 - `omp for` construct
 - `omp sections` construct
 - `omp task` construct
- Each of them **automatically** divides work among threads
- Work sharing is important for the improving parallel efficiency:
 - Work sharing divides large tasks, like loop iterations or sections of code, among multiple threads, ensuring all threads are actively performing useful work to maximize CPU usage
 - However, improper work sharing may introduce unbalanced load as some threads may end up doing more work than other threads
 - Good news: OpenMP allows the distribution of tasks dynamically or statically to balance the load among threads, ensuring that no single thread becomes a bottleneck

The **omp for** Directive

```
omp_set_num_threads(3);  
#pragma omp parallel for  
    for(i = 0; i < 12; i++)  
        c[i] = a[i] + b[i];
```

- Threads are assigned an independent set of iterations that do not overlap each other
 - Thread 0: $i = \{0, 1, 2, 3\}$
 - Thread 1: $i = \{4, 5, 6, 7\}$
 - Thread 2: $i = \{8, 9, 10, 11\}$
- Threads must wait at the end of work-sharing construct



[pic above assumes a team of **three** threads]

The “for” OpenMP Construct: different ways to invoke it

- In many cases, you may not just want to parallelize a for-loop in a parallel region
 - Ex: you may have four loops to parallelize in a parallel region

The diagram consists of two rectangular boxes, one on the left and one on the right, each containing OpenMP code. Red arrows originate from the closing curly braces of the parallel regions in both boxes and point towards a central point located above a text box at the bottom of the slide. This visualizes the concept of an implicit barrier at the end of a parallel region.

```
#pragma omp parallel
{
    #pragma omp for
    for ( int i=0; i< MAX; i++) {
        res1[i] = huge1();
    }

    #pragma omp for
    for ( int i=0; i< MAX; i++) {
        res2[i] = huge2();
    }
}

#pragma omp parallel for
{
    for (int i=0; i< MAX; i++) {
        res1[i] = huge1();
    }
}

#pragma omp parallel for
{
    for (int i=0; i< MAX; i++) {
        res2[i] = huge2();
    }
}
```

There is an implicit barrier at the end of the parallel region “}”.

What `for` loops can be parallelized?

- OpenMP will only parallelize `for` loops that are in **canonical form**
 - Part I: where should the loop start?
 - Part II: where should the loop end?
 - Part III: how should the loop proceed with what step size?

$$\text{for} \left(\begin{array}{l} \text{index} = \text{start} \quad ; \quad \begin{array}{l} \text{index} < \text{end} \\ \text{index} \leq \text{end} \\ \text{index} \geq \text{end} \\ \text{index} > \text{end} \end{array} \quad ; \quad \begin{array}{l} \text{index} ++ \\ ++\text{index} \\ \text{index} -- \\ --\text{index} \\ \text{index} += \text{incr} \\ \text{index} -= \text{incr} \\ \text{index} = \text{index} + \text{incr} \\ \text{index} = \text{incr} + \text{index} \\ \text{index} = \text{index} - \text{incr} \end{array} \end{array} \right)$$

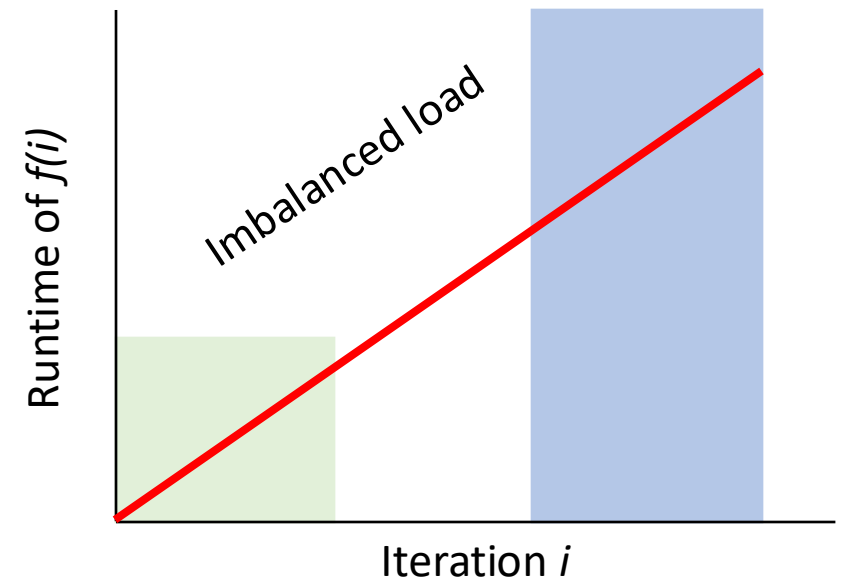
Default **parallel for** Partitioning Algorithm

- By default, OpenMP partitions iterations evenly across working threads
 - Each thread is assigned with `std::ceil(n/thread_count)` iterations
 - This partition works well if the per-iteration work is evenly distributed
- However, this may lead to *load imbalance* when the per-iteration work varies
 - Assume the time required by a call to $f(i)$ is **proportional** to i

```
for(i = 0; i <= n; i++)  
    x[i] = f(i);
```

- Earlier partitions have much less work than later partitions

Challenge: What would be a good partitioning strategy to distribute iterations across threads to balance the load of threads (i.e., no single thread becomes the bottleneck)?



Solution for Load Balancing Optimization: `schedule` Clause

- The `schedule` clause defines how loop iterations are scheduled across threads

`schedule(static [,chunk])`

- Each thread grabs `chunk` iterations in a round-robin fashion (a round means $n_threads * chunk$)
- Low scheduling overhead, may cause load imbalance

`schedule(dynamic[,chunk])`

- Each thread grabs `chunk` iterations and request the next `chunk` iterations when done in no particular order
- Higher scheduling overhead (e.g., `chunk = 1`), can reduce load imbalance

`schedule(guided[,chunk])`

- Threads are initially assigned large chunks of iterations; as threads complete their chunks, subsequent chunks become smaller but no smaller than `chunk` though
- Decent scheduling overhead, can balance the load in most applications

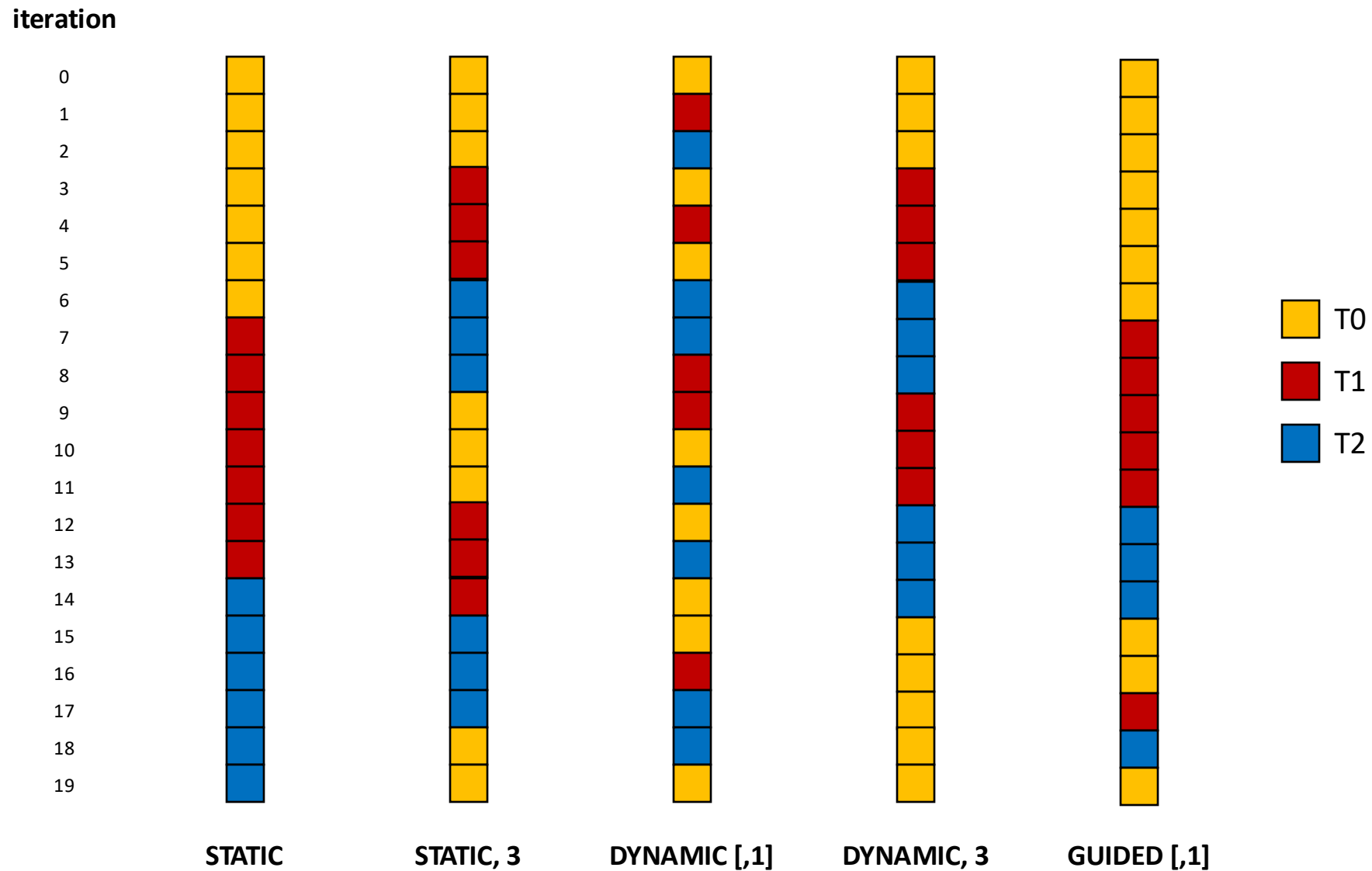
schedule Clause Example

```
#pragma omp parallel for schedule(static, 8)
for( int i = 0; i <= end; i += 1 )
{
    do_something(i);
}
```

```
#pragma omp parallel for schedule(dynamic, 2)
for( int i = 0; i <= end; i += 1 )
{
    do_something(i);
}
```

```
#pragma omp parallel for schedule(guided, 1)
for( int i = 0; i <= end; i += 1 )
{
    do_something(i);
}
```


Loop Scheduling (Assume 3 Threads)



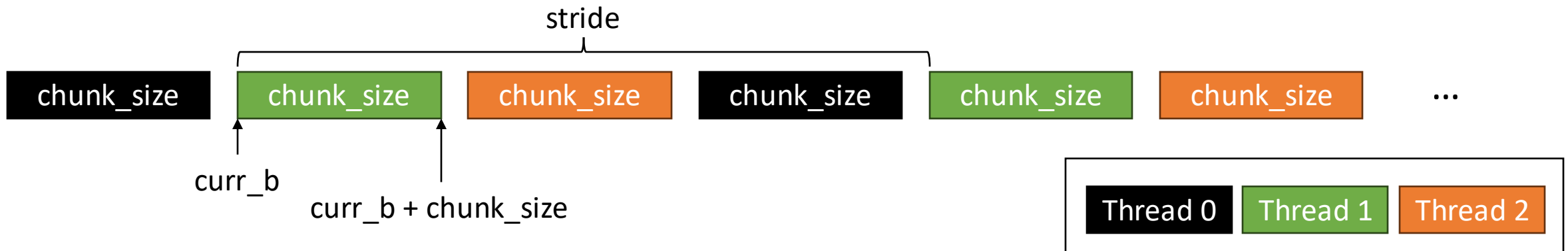
Choosing a Schedule

- `STATIC` best if you know you have balanced loops – least scheduling overhead
- `STATIC, n` good for loops with mild or smooth load imbalance
 - The round-robin strategy allows each thread to keep data locality to certain extent
- `DYNAMIC` useful if iterations have widely varying loads
 - Prone to adversely impact data locality because which thread grabs which chunk is non-deterministic
- `GUIDED` often less expensive than `DYNAMIC`
 - Beware of loops where first iterations are the most expensive

Static Scheduling Algorithm: `static, chunk_size`

- Divide iterations into chunks and distributes chunks to threads in a round-robin order

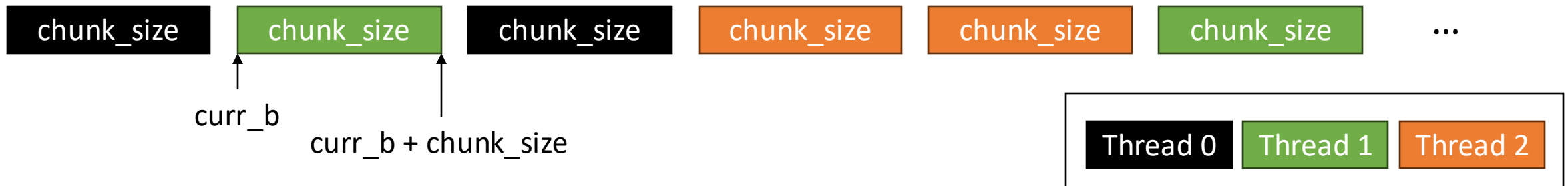
```
void loop(size_t N, size_t W, size_t curr_b, size_t chunk_size, F&& func) {  
    // defining the stride size for each round  
    size_t stride = W * chunk_size;  
    // each partition range is indicated by [curr_b, curr_e)  
    while(curr_b < N) {  
        size_t curr_e = std::min(curr_b + chunk_size, N);  
        func(curr_b, curr_e);  
        curr_b += stride;  
    }  
}
```



Dynamic Scheduling Algorithm: `dynamic, chunk_size`

- Split iterations into many partitions each of size equal to the given chunk size
 - Using an atomic variable `next` to indicate the next starting position of unassigned iterations

```
void loop(size_t N, std::atomic<size_t>& next, F&& func) {  
    // next is an atomic variable that indicate the next partition's position  
    size_t chunk_size = (given_chunk_size == 0) ? size_t{1} : given_chunk_size;  
    size_t curr_b = next.fetch_add(chunk_size);  
    // each partition range is indicated by [curr_b, std::min(curr_b + chunk_size, N))  
    while(curr_b < N) {  
        func(curr_b, std::min(curr_b + chunk_size, N));  
        curr_b = next.fetch_add(chunk_size);  
    }  
}
```



Guided Scheduling Algorithm: `guided, chunk_size`

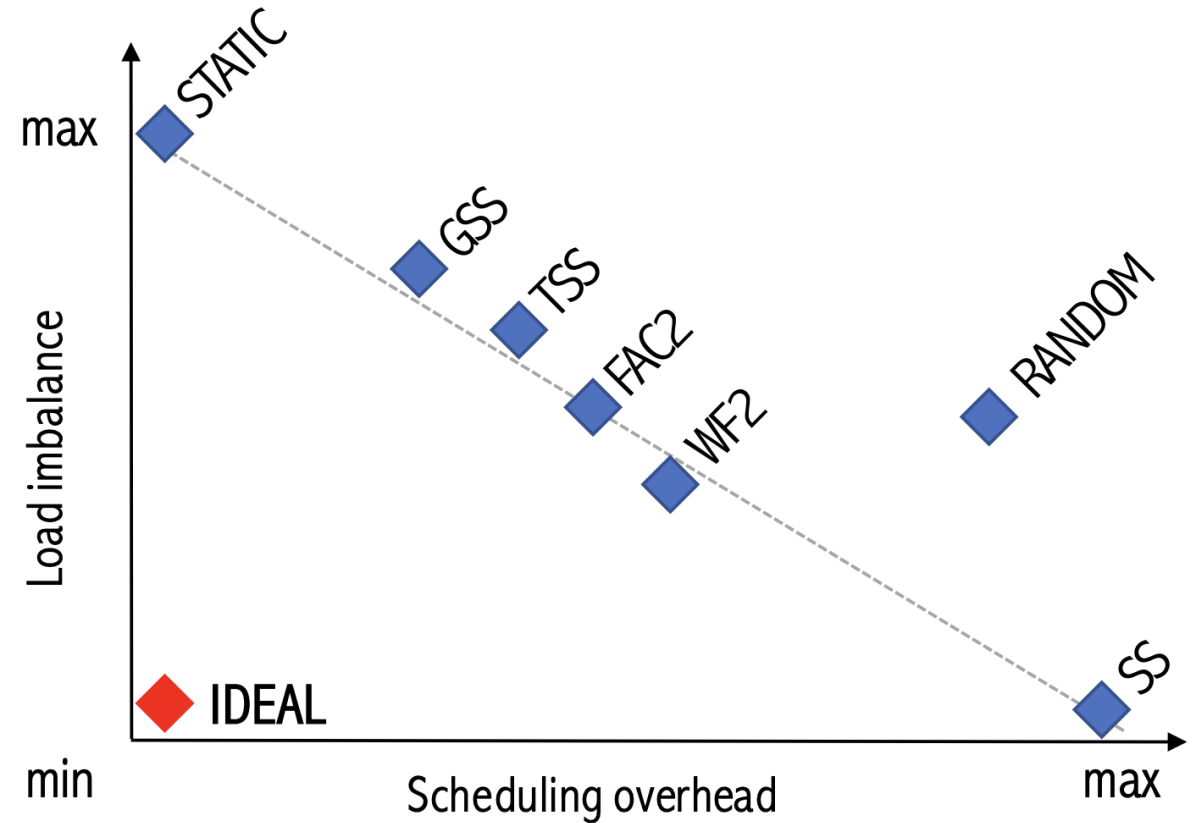
- The size of a partition is proportional to the number of unassigned iterations divided by the number of workers, and the size will gradually decrease to the given chunk size
 - The last partition may be smaller than the chunk size – need to do a `std::min` check

```
size_t p1 = 2 * W * (chunk_size + 1);
float p2 = 1 / static_cast<float>(W);
size_t curr_b = next.load();
while(curr_b < N) {
    size_t r = N - curr_b;
    // fine-grained iterations (same as dynamic sched)
    if(r < p1) {
        while(1) {
            curr_b = next.fetch_add(chunk_size);
            if(curr_b >= N) {
                return;
            }
            func(curr_b, std::min(curr_b + chunk_size, N));
        }
        break;
    }
}
```

```
// coarse-grained iterations
else {
    size_t q = static_cast<size_t>(p2 * r);
    if(q < chunk_size) {
        q = chunk_size;
    }
    size_t curr_e = std::min(curr_b + q, N);
    if(next.compare_exchange_strong(curr_b, curr_e))
    {
        func(curr_b, curr_e);
        curr_b = next.load();
    }
} // end of while loop
```

Other Loop Scheduling Algorithms

- Dynamic self-scheduling (SS)
- Guided self-scheduling (GSS)
- Trapezoid self-scheduling algorithm (TSS)
- Factoring (FAC)
- Adaptive factoring (FAC2)
- Weighted factoring (WF)
- Adaptive weighted factoring (WF2)
- Taper
- Random
- ... (learn more on [OpenMP Loop Scheduling Revisited: Making a Case for More Schedules](#))



The OpenMP **collapse** clause

- Background: you have two nested for loops
 - Outer loop, 10 trips;
 - Inner loop, 1,000,000 trips
 - You have 32 threads
- OpenMP parallelizing outer loop not a good idea
 - Only 10 of the 32 threads get to do work
- OpenMP parallelizing inner loop ok
 - Miss the opportunity to parallelize the outer 10 trips

OpenMP collapse

- You can use OpenMP collapse construct to collapse nested loops and then parallelize the resulting 1D iterations using different scheduling strategies (static, dynamic, guided)
 - Of course, you can also choose to do this on your own
- Caveats:
 - The bounds of the collapsed for loops should be fixed and available when the execution reaches the embedded loops
 - You cannot change the number of iterations in the inner loop at different iterations in the outer loop
 - You cannot have data dependencies between the inner and outer loops

```
for (i = 0; i < N; i++){  
    b[i] = 0.;  
    for (j = 0; j < N; j++)  
        b[i] += A[i][j] * x[j]  
}
```


Example of OpenMP **collapse**

Sum up all elements in the matrix arr

```
#include <cstdlib>
#include <chrono>
#include <iostream>
#include <omp.h>

using std::chrono::high_resolution_clock;
using std::chrono::duration;
using std::cout;

float reduce(const float* arr, const size_t m, const size_t n) {
    float sum = 0.0;
    #pragma omp parallel for collapse(2)
    for (size_t i = 0; i < m; i++) {
        for (size_t j = 0; j < n; j++) {
            #omp atomic
            sum += arr[i * n + j];
        }
    }
    return sum;
}
```

```
int main(int argc, char* argv[]) {
    size_t m = std::atoi(argv[1]);
    size_t n = std::atoi(argv[2]);
    size_t t = std::atoi(argv[3]);
    float *arr = new float[m*n];

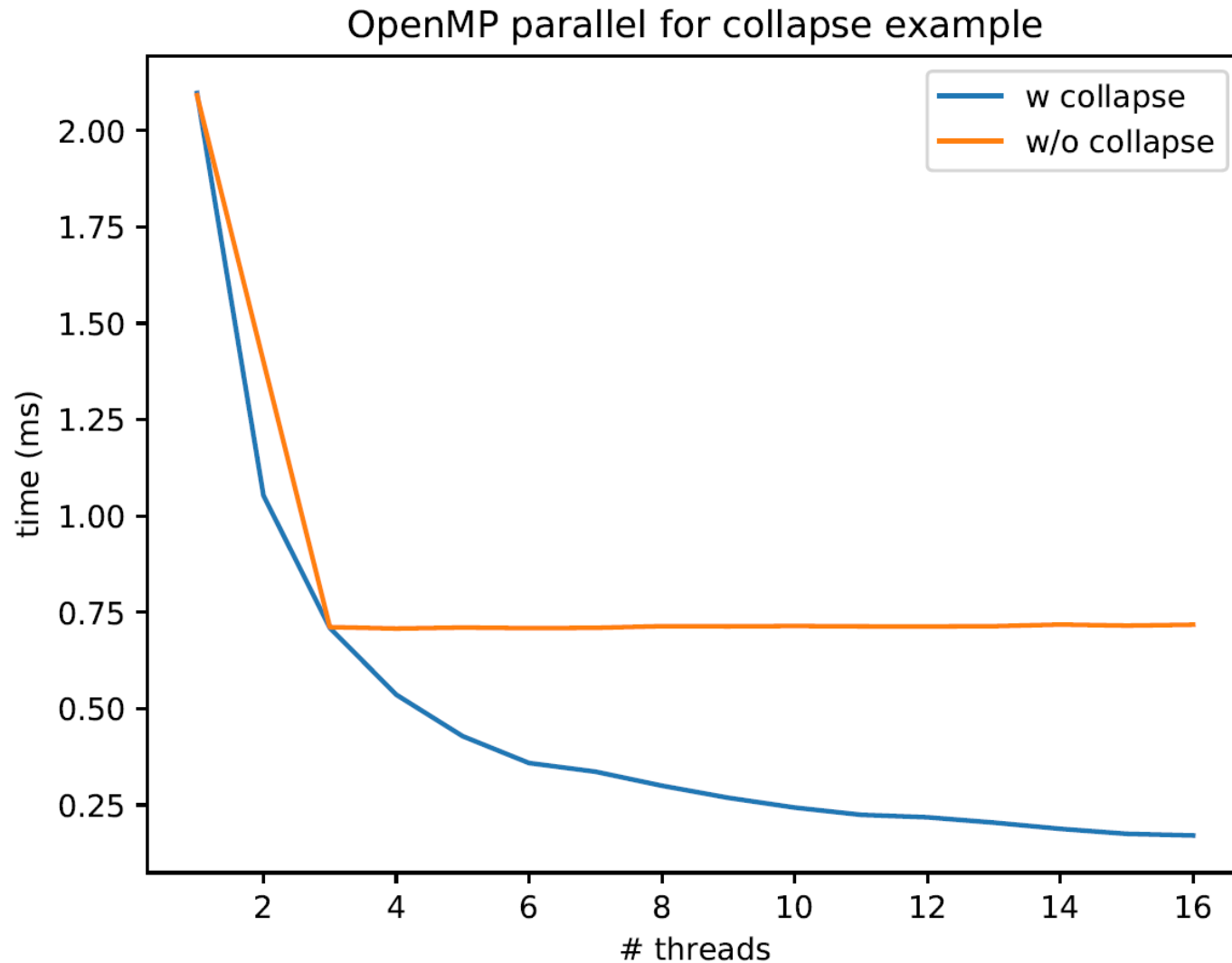
    #pragma omp parallel for
    for (size_t i = 0; i < m; i++) {
        for (size_t j = 0; j < n; j++) {
            arr[i * n + j] = 1.1f;
        }
    }

    high_resolution_clock::time_point start;
    high_resolution_clock::time_point end;
    duration<double, std::milli> duration_sec;

    float res;
    omp_set_num_threads(t);
    start = high_resolution_clock::now();
    for (size_t i = 0; i < 10; i++) { // Run reduce 10 times, to get a good average time
        res = reduce(arr, m, n);
    }
    end = high_resolution_clock::now();
    duration_sec = std::chrono::duration_cast<duration<double, std::milli>>(end - start) / 10.;

    cout << "Result: " << res << "\n";
    cout << t << " " << duration_sec.count() << "\n";
    delete[] arr;
    return 0;
}
```

Benefit of Collapsing Nested Loops



8-core AMD Processor, one NUMA node
Test run with $m = 3$, $n = 1E6$, $t = 1$ to 16.

```
float sum = 0.0;
#pragma omp parallel for collapse(2)
for (size_t i = 0; i < m; i++) {
    for (size_t j = 0; j < n; j++) {
        #omp atomic
        sum += arr[i * n + j];
    }
}
```

Quiz: Collapsing Loop using OpenMP `collapse`

- Consider the following parallel matrix multiplication code:

```
void matrix_multiply(int N, double A[N][N], double B[N][N], double C[N][N]) {  
    #pragma omp parallel for collapse(3)  
    for (int i = 0; i < N; i++) {  
        for (int j = 0; j < N; j++) {  
            C[i][j] = 0;  
            for (int k = 0; k < N; k++) {  
                C[i][j] += A[i][k] * B[k][j];  
            }  
        }  
    }  
}
```

- Is this code correct? If not, how can we fix it?